

UNHEARD VOICES

Project ID: Cs29

Project Supervisor: Dr. Nora Niazy

Assistant Supervisor: Eng. Ahmed Fouad

2024

Proposed System for Unheard Voices Application

Khaled Mohamed Nagy
Software Engineer
Department of Computer
Science in Maadi, Egypt
khaledmohammednagy2014@gmail.com
[mail.com](mailto:khaledmohammednagy2014@gmail.com)

Amr Sabry Mohamed
Software Engineer
Department of Computer
Science in Maadi, Egypt
mrw9293@gmail.com

Osama Gamal Hamed
Software Engineer
Department of Computer
Science in Maadi, Egypt
osamagamal9988@gmail.com

Abstract —*Unheard Voices is a comprehensive application and website designed to facilitate seamless communication between deaf individuals and those who are not fluent in sign language. This innovative tool utilizes computer vision and machine learning algorithms to recognize and translate sign language into text, and vice versa. The application features a camera-based interface that captures and interprets sign language gestures, converting them into readable text. Additionally, it can convert spoken voice into text, allowing users to read and understand spoken language. Furthermore, the tool includes a sign language learning section that provides users with a library of sign language GIFs, enabling them to learn and practice sign language. This project aims to bridge the communication gap between deaf and hearing individuals, promoting greater accessibility and inclusivity in daily interactions.*

Keywords-

RNN-Recurrent neural network

ML-Machine Learning

ASL-American Sign Language

LSTM-Long Short Term Memory

I. INTRODUCTION

Sign language is a visual-gestural language used by the deaf and hard of hearing community to communicate. It is a complex and rich language that involves the use of handshapes, orientation, movement, and facial expressions to convey meaning. Sign languages are not universal and vary across different regions and cultures. Each sign language has its own grammar,

vocabulary, and syntax, and is often used in conjunction with spoken languages.

The American Sign Language (ASL) dataset is a comprehensive collection of videos that represent the signs and gestures used in ASL. This dataset is particularly useful for developing systems that can recognize and translate ASL into spoken languages.

The ASL dataset is designed to help bridge the communication gap between the deaf and hearing communities by providing a standardized and accessible platform for learning and using ASL. It is an essential tool for developing intelligent computer vision systems that can recognize and interpret ASL signs, enabling more effective communication and interaction between individuals with different abilities.

II. RELATED WORK

Over the past two decades, researchers have employed various machine learning techniques to recognize American Sign Language (ASL). This effort aims to bridge the communication gap between the deaf community and the general public by developing a real-time sign language translator.

Several studies have achieved notable results in ASL recognition. For instance, Brandon Garcia and Sigberto Alarcon Viesca developed a robust model for letters "a-e" and a modest one for letters "a-k" using the Surrey and Massey datasets on ASL and the GoogleNet architecture. Their system achieved a validation accuracy of nearly 98% for five letters and 74% for ten letters.

Another study by Hardie Cate, Fahim Dalvi, and Zeshan Hussain employed machine learning techniques for temporal classification, specifically the multivariate case. Although the results were not exceptionally high, they believe that a more efficient implementation of the algorithms can yield more complex models that will perform better. They used the University of South Wales dataset, which contains 95 unique signs, and achieved an accuracy of 78.6% on a high-quality dataset.

These studies demonstrate the progress made in ASL recognition using various machine learning techniques, highlighting the importance of this research in facilitating communication between the deaf community and the public[1].

III. APPROACH AND METHODS

A. RNN

Recurrent Neural Networks (RNNs) are a type of artificial neural network that are particularly useful for processing sequential data such as time series data, natural language processing, and speech recognition. Unlike traditional neural networks, RNNs are designed to retain information from previous inputs to influence the current output. This ability to maintain a hidden state allows RNNs to capture long-term dependencies in sequential data, making them suitable for tasks such as language translation, text generation, and stock market prediction[2].

B. Why RNN?

The use of Recurrent Neural Networks (RNNs) over Convolutional Neural Networks (CNNs) in sign language recognition modules is primarily due to the following reasons:

Temporal Dependencies: Sign languages involve complex temporal dependencies between gestures, which are difficult to capture using CNNs. RNNs are designed to handle sequential data and can learn these temporal dependencies more effectively.

Sequence Learning: RNNs are well-suited for sequence learning, which is crucial in sign language recognition. They can learn the patterns and relationships between gestures in a sequence, allowing for more accurate recognition.

Handling Variability: Sign languages involve a lot of variability in terms of hand movements, orientation, and other factors. RNNs are more robust to these variations and can learn to recognize sign language gestures more accurately.

Dynamic Gesture Recognition: RNNs are better suited for dynamic gesture recognition, which involves recognizing gestures that change over time. This is particularly important in sign language recognition, where gestures can be dynamic and complex.

Handling Unsegmented Data: Sign language recognition often involves unsegmented video streams, which can be challenging for CNNs. RNNs can handle this type of data more effectively by learning to recognize gestures in a sequence.

Improved Performance: RNNs have been shown to achieve better performance than CNNs in sign language recognition tasks. This is because RNNs can learn more complex patterns and relationships between gestures, leading to more accurate recognition.

Handling Complex Gestures: Sign languages involve complex gestures that require the recognition of multiple hand movements and orientations. RNNs are better suited to handle these complex gestures by learning to recognize the patterns and relationships between them.

Real-Time Processing: RNNs are designed for real-time processing, which is crucial in sign language recognition applications where fast and accurate recognition is required[2].

C. LSTM

LSTM stands for Long Short-Term Memory, and it is a type of recurrent neural network (RNN) architecture.

LSTM networks contain special units called memory cells, which can store information for long durations of time. Making them well-suited for learning from sequences of data.

Applications of LSTM networks include:

Time series prediction and forecasting tasks like stock price prediction and weather forecasting,

Speech recognition and synthesis, Handwriting recognition, Gesture recognition in sign language, Less Data Required, Real-Time Processing, Robustness to Noise and Variability, Adaptability and Flexibility

Overall, using LSTM networks for sign language recognition offers the potential to develop robust, accurate, and real-time systems for facilitating communication and interaction between hearing-impaired individuals and the rest of society[3].

D. OpenCV

OpenCV (Open Source Computer Vision Library) is a library of programming functions mainly for real-time computer vision. It was originally developed by Intel and is now maintained by the OpenCV Foundation. OpenCV is a cross-platform library, meaning it can run on various operating systems such as Windows, Linux, macOS, and mobile platforms like Android and iOS.

OpenCV can be used for various tasks in the sign-to-text module, including:

Image Processing: process the images of the sign language gestures. This includes tasks such as: Image Filtering to enhance or remove noise from images, Applying Thresholding techniques to segment the images based on the intensity or color, detecting edges in the images which can help in identifying the contours of the sign language gestures also detect the objects(hands) and track their movement, also can be used to train machine learning models using the preprocessed images and the corresponding text labels[5].

E. MediaPipe

MediaPipe is a framework developed by Google that enables developers to build sign language recognition solutions into their apps. It provides a set of tools and APIs for processing and analyzing hand gestures, facial features, and other visual cues to recognize sign language gestures and convert them to text in real-time. Once the sign language gestures are recognized, MediaPipe can convert them to text in real-time. This is done by using machine learning models that map the recognized gestures to corresponding text[6].

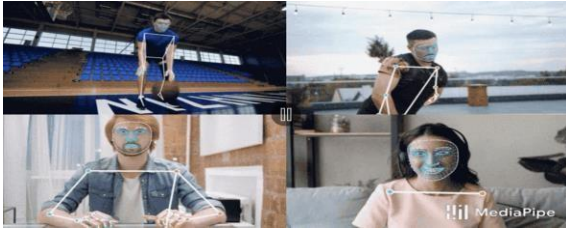


Figure 3. Recognized landmarks using Media Pipe library.

F. Tensorflow

TensorFlow is a popular open-source machine learning framework used for building and training machine learning models. It is widely used for various applications, including sign language detection. TensorFlow is used to build and train machine learning models for sign language detection. TensorFlow's transfer learning capabilities allow models to be trained on large datasets and then fine-tuned for specific tasks, such as sign language detection. This approach can significantly improve the accuracy and performance of the models, can be used to collect and preprocess datasets for sign language detection. This includes collecting images and videos of sign language gestures, labeling them, and then using the data to train and fine-tune machine learning models[7].

G. Flutter

Flutter is a cross-platform framework for building mobile, web, and desktop applications. It uses the Dart programming language and provides a rich set of tools for building UI components, so it will be used for building the mobile application includes a user friendly communication page that will help deaf people in their daily communication with the other people[8].

H. Java Script

JavaScript is known for its flexibility and ability to manipulate web pages dynamically. It is widely used for building web applications, mobile apps, and desktop applications, it will be used to build a web site to help deaf in communication that can be accessed across the internet. Its flexibility and ability to manipulate web pages dynamically make it an ideal choice for creating interactive and responsive interfaces[9].

IV. COMPONENTS AND FEATURES

A. Sign to text

The sign-to-text process involves capturing hand gestures using a camera and recognizing them through a machine learning model trained on sign language data. The model identifies the gestures and converts them into corresponding text in real-time. This process leverages computer vision and deep learning techniques.

B. Text to speech

After sign language is translated to text, the text-to-speech (TTS) process begins. TTS uses natural language processing and synthesis techniques to convert written text into spoken words in different

languages. A TTS engine analyzes the text's linguistic and phonetic properties and then generates human-like speech. This allows the translated sign language text to be audibly communicated, enabling seamless interaction.

C. Speech to text

The speech-to-text process involves converting spoken language into written text, by analyzing audio input, identifying phonetic patterns and transcribing them into text in real-time. Deaf individuals can use the microphone to receive spoken communication as text on their devices, facilitating their understanding and interaction in conversations where spoken language is used. This allows them to read and respond to spoken dialogue effectively.

D. Text to GIF

The text-to-GIF process involves translating written text into corresponding animated images (GIFs) that represent words or phrases in sign language. This approach is beneficial for deaf individuals who may not be proficient in reading but understand sign language visually. By converting text to GIFs, users can quickly grasp the meaning through familiar signs, enhancing comprehension and communication efficiency.

E. Learn Section

In the learning section, users explore categories with words represented by GIFs of their signs. Each word's page displays the word and its sign language GIF for visual learning. Users can open the camera to practice and test their sign, receiving real-time feedback from a comprehensive dataset. This interactive approach combines visual aids with practical testing for an engaging and effective learning experience.

F. Translation:

You can use the application and website in both Arabic and English. This allows you to have conversations in either language. The app can convert sign language into text in both Arabic and English, and it can also convert Arabic or English text into GIF sign language.

V. ARCHETECTURE DIAGRAM

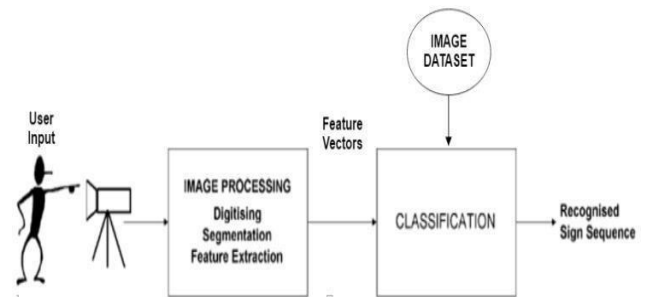


Figure 3. Architecture diagram for our proposed system

The architecture depicts that it is a real time sign language recognition system and also the methods for image processing is mentioned in the paper. The proposed image dataset is the Massey University dataset because it contains an exhaustive set of data.

VI. DATASET

A. Google - Isolated Sign Language Recognition:

The Isolated Sign Language Recognition corpus from kaggle (version 1.0) is a collection of hand and facial landmarks generated by Mediapipe version 0.9.0.1 on ~100k videos of isolated signs performed by 21 Deaf signers from a 250-sign vocabulary.

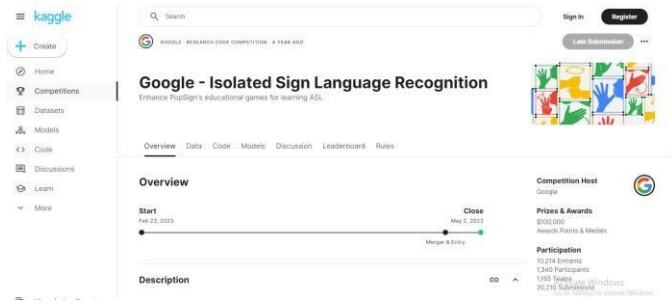


Figure 1 ASL Dataset on Kaggle website

B. Data Collection:

We created a dataset for this project to strengthen the previous dataset with important 50-sign vocabulary to create conversations. The new dataset that was created consists of 500 video samples for each sign, capturing various sign language gestures. Each video is recorded using a camera and processed frame-by-frame to ensure comprehensive coverage of the sign.

C. Landmark Extraction:

For each frame in the video, landmarks are extracted using MediaPipe, a powerful library for detecting facial, pose, and hand landmarks. These landmarks provide precise coordinates for key points on the face, body, and hands, essential for accurately recognizing signs.

D. Data Structuring:

The extracted landmarks from each frame are structured into a pandas DataFrame, categorizing them into types such as face, right hand, left hand, and pose. This structured format allows for efficient processing and analysis.

E. Ordinal Encoding:

Each sign is assigned an ordinal number using ordinal encoding, creating a mapping between the sign names and numerical codes. This encoding facilitates the training and prediction process by converting categorical data into numerical format.

F. Model Integration:

The structured landmark data is converted into a format compatible with a pre-trained TensorFlow Lite model. The model is used to predict the signs based on the landmarks, providing real-time sign language recognition.

G. Real-time Processing:

The dataset enables real-time processing by capturing and analyzing frames. Each frame is processed to extract landmarks and predict the sign, allowing for immediate feedback and interaction.

H. Comprehensive Dataset:

The dataset covers a wide range of signs, ensuring robustness and accuracy in the sign-to-text translation. The extensive data collection and processing methods ensure that the model is well-trained and capable of handling diverse sign language gestures.

VII. IMPLEMENTATION

A. Dataset Preprocessing

Each video here consists of 15 frames, and each frame consists of 543 landmarks. Each landmark is represented in 3D coordinates (x, y, z). Append the landmarks from each video to the next video until you have processed all 500 videos. Store the combined landmarks in a form array in a variable X. The labels for each video should be saved in a variable y.

B. Model Architecture

Our proposed system leverages a deep learning model built with TensorFlow and MediaPipe. The architecture of the model includes several components:

1. Input Layer: Defines the input shape as (15, 1629).
2. First LSTM Layer: Add an LSTM layer with 64 units, return sequences, ReLU activation, and an input shape of (15, 1629).
3. Second LSTM Layer: Add an LSTM layer with 128 units, return sequences, and ReLU activation.
4. Third LSTM Layer: Add an LSTM layer with 64 units, no return sequences, and ReLU activation.
5. First Dense Layer: Add a dense layer with 64 units and ReLU activation.
6. Second Dense Layer: Add a dense layer with 32 units and ReLU activation.
7. Output Dense Layer: Add a dense layer with a number of units equal to actions.shape[0] and softmax activation.

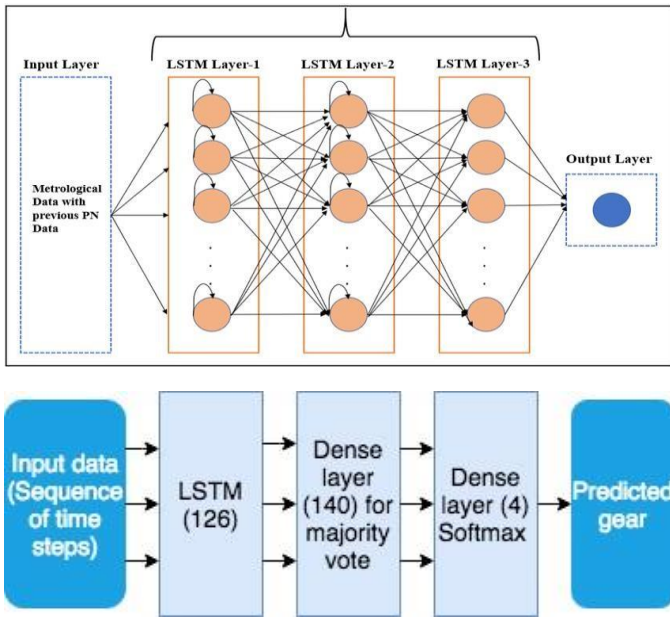


Figure.2 Network architecture for the LSTM mode[4]

C. Training the Model

1. Data Preparation: It loads sequences of hands and face gesture data from `.npy` files, reshapes them in array, and labels them according to predefined actions.
2. Data Splitting: It splits the dataset into training and testing sets, with 90% of the data used for training and 10% for testing.
3. Model Creation: It defines a Sequential LSTM model with three LSTM layers followed by Dense layers, designed to classify the gestures.
4. Model Training: It compiles the model with the Adam optimizer and categorical cross-entropy loss, then trains it for 100 epochs.
5. Test and Evaluation: It predicts labels for the test set and calculates the accuracy of the model predictions which is estimated at 0.9975

D. Real-time Detection of Sign language

In the final step, our system will process video frames from a webcam in real time to detect and classify hand gestures. The process includes:

1. Video Capture: Using OpenCV, we will capture a live video feed from the webcam.
2. Landmark Detection: Each frame will be processed using the MediaPipe holistic model to detect and extract key landmarks.
3. Visualization: We will draw landmarks on the video feed for visual feedback. Landmarks for the face, pose, and hands will be drawn using specific styles.
4. Sequence Analysis: The extracted key points will be stored and analyzed in sequences of 15 frames. If a valid sequence is detected, the TensorFlow Lite interpreter will predict the gesture.

Prediction and Display: The predicted gesture will be displayed on the video feed, providing real-time feedback to the user. A confidence threshold will ensure only reliable predictions are shown.

This comprehensive approach ensures that our model is not only accurate but also efficient and capable of running in real time on various devices.

E. Test and Evaluation

1. Prediction: Use the model to predict labels for the test data X_{test} and store the predictions in y_{hat} .
2. True Labels: Convert the true labels y_{test} from a one-hot encoded format to a list of class indices and store them in y_{true} .
3. Predicted Labels: Convert the predicted labels y_{hat} from a one-hot encoded format to a list of class indices.
4. Accuracy Calculation: Calculate and print the accuracy score by comparing the true labels y_{true} with the predicted labels y_{hat} .
5. The accuracy of the model predictions which is estimated at 0.9975.

VIII. CONCLUSION

The "Unheard Voices" project has significantly improved communication between deaf and hearing individuals through innovative software. Key achievements include real-time conversations via sign language recognition, voice-to-text translation, and text-to-sign GIF conversion, the software also includes sections for learning sign language, allowing hearing users to practice and receive feedback, and promoting sign language literacy. Additionally, the ability to make a conversation using my hands to break the barriers between deaf and hearing people.

IX. ACKNOWLEDGMENTS

We extend our gratitude to Modern Academy and its board for their support throughout our educational journey. Special thanks to Prof. Dr. Mahmoud Gadallah for his guidance and the department staff for their assistance. We are grateful to Dr. Nora Niazy for her invaluable guidance and encouragement, and to Eng. Ahmed Fouad for his patience and support. Their contributions were crucial in bringing our project to fruition, and we hope it benefits many people.

X. REFERENCES

- [1] https://www.researchgate.net/publication/380564751_Sign_Language_Recognition_Using_Machine_Learning
- [2] https://openaccess.thecvf.com/content_cvpr_2017/papers/Cui_Recurrent_Convolutional_Neural_CVPR_2017_paper.pdf
- [3] <https://www.sciencedirect.com/topics/computer-science/long-short-term-memory-network>
- [4] https://www.researchgate.net/figure/Sequential-Long-Short-Term-Memory-LSTM-architecture_fig4_355337314
- [5] <https://en.wikipedia.org/wiki/OpenCV>
- [6] <https://developers.googleblog.com/en/signall-sdk-sign-language-interface-using-mediapipe-is-now-available-for-developers/>
- [7] <https://www.geeksforgeeks.org/sign-language-recognition-system-using-tensorflow-in-python/>
- [8] <https://flutter.dev/>
- [9] <https://en.wikipedia.org/wiki/JavaScript>